Programmation Orientée Objet (POO) à destination de la robotique

Travaux Pratique : Arguments et Logs

Enseignant : Adam GOUGUET
Mail : adam.gouguet@imt-nord-europe.fr
Année : 2025–2026

Table des matières

1 Objectifs du TP **2**

2 Contexte **2**

3 Partie I : Arguments en ligne de commande **2**
3.1 Pourquoi utiliser des arguments 2
3.2 Mise en place de `argparse` 2
3.3 Sélection dynamique du moteur 3

4 Partie II : Logs et Debug **3**

5 Pourquoi ne pas utiliser `print()` **3**

6 Découverte du module `logging` **3**
6.1 Niveaux de logs 4

7 Configuration des logs **4**

8 Logs et arguments **4**

9 Instrumentation du projet **4**
9.1 Robot 4
9.2 Moteur 5
9.3 Contrôleurs 5

10 Logs dans un fichier **5**

11 Travail attendu **5**

12 Pour aller plus loin... **5**

1 Objectifs du TP

À l'issue de ce TP, vous serez capables de :
— utiliser des arguments en ligne de commande avec `argparse`,
— configurer dynamiquement le comportement d'un programme,
— sélectionner des composants logiciels via des options,
— comprendre l'importance des logs dans un système robotique,
— utiliser le module `logging` de Python,
— instrumenter un logiciel pour le debug et la supervision.

Ce TP vous place dans une situation proche d'un logiciel de robotique réel, utilisable, configurable et observable sans modifier le code source.

2 Contexte

Dans les TP précédents, vous avez développé un simulateur de robot mobile. Jusqu'à présent :
— les paramètres étaient codés en dur,
— le comportement était observé principalement via l'affichage graphique,
— le debug reposait sur quelques `print()`.

Dans un contexte professionnel :
— un programme doit être configurable au lancement,
— le comportement doit pouvoir être modifié sans toucher au code,
— l'état interne du système doit être observable.

Ce TP aborde deux outils essentiels pour répondre à ces contraintes :
— les **arguments en ligne de commande**,
— les **logs**.

3 Partie I : Arguments en ligne de commande

Les arguments en ligne de commande permettent de configurer un programme au moment de son lancement.

En Python, le module standard `argparse` est utilisé pour cette tâche.

3.1 Pourquoi utiliser des arguments

Les arguments permettent notamment de :
— activer ou désactiver des fonctionnalités,
— choisir un type de moteur ou de contrôleur,
— modifier des paramètres sans recompiler ni modifier le code.

Exemple d'exécution attendue :

```
python main.py --moteur differentiel
python main.py --moteur omnidirectionel
```

3.2 Mise en place de `argparse`

Dans le fichier principal de votre programme, vous devez utiliser `argparse` pour définir les options suivantes :

Arguments obligatoires ou optionnels

- `-moteur` : type de moteur à utiliser
  - `differentiel`
  - `omni`
- `-dt` : pas de temps de la simulation (optionnel)

Exemple

À faire

- analyser les arguments au lancement du programme,
- afficher une erreur claire si un moteur inconnu est demandé,
- créer le bon objet moteur en fonction de l'option choisie.

3.3 Sélection dynamique du moteur

Votre code ne doit **pas** contenir de moteur codé en dur.

À faire

- utiliser l'argument `-moteur` pour choisir la classe moteur,
- instancier dynamiquement le moteur correspondant,
- préparer le projet à l'ajout futur de nouveaux moteurs.

4 Partie II : Logs et Debug

Une fois le programme configurable, il est nécessaire de pouvoir observer son comportement interne.

Les logs permettent de :

- comprendre ce que fait le programme,
- détecter des erreurs,
- analyser des comportements anormaux.

5 Pourquoi ne pas utiliser `print()`

La fonction `print()` est utile pour des tests rapides, mais elle présente de nombreuses limites :

- impossible de filtrer les messages,
- difficile à désactiver proprement,
- aucun niveau de gravité,
- peu adaptée aux systèmes complexes.

Python fournit un module standard dédié à la journalisation : `logging`.

6 Découverte du module `logging`

Le module `logging` permet d'émettre des messages associés à un niveau de gravité.

6.1 Niveaux de logs

Les principaux niveaux sont :
- `DEBUG` : informations détaillées (développement),
- `INFO` : fonctionnement normal,
- `WARNING` : situation anormale non bloquante,
- `ERROR` : erreur empêchant une fonctionnalité,
- `CRITICAL` : erreur critique.

7 Configuration des logs

Créez un fichier `logging_config.py` dans votre package.

À faire
- définir un format de log lisible,
- afficher la date, le niveau, le module et le message,
- définir le niveau par défaut à `INFO`.

Exemple de format attendu

```
[2026-01-15 10:42:31] INFO robot.moteur : Commande reçue
```

8 Logs et arguments

Vous devez maintenant lier les arguments et les logs.

À faire
- ajouter une option `-debug`,
- activer les logs de niveau `DEBUG` si l'option est présente,
- ne jamais modifier le niveau de logs directement dans le code.

Exemple

```
python -m robot --moteur differentiel --debug
```

9 Instrumentation du projet

Ajoutez des logs dans les composants suivants :

9.1 Robot

- création du robot,
- mise à jour de la position et de l'orientation,
- niveau `DEBUG`.

9.2 Moteur

— commandes reçues,
— commandes invalides (`WARNING`),
— erreurs critiques (`ERROR`).

9.3 Contrôleurs

— commandes générées,
— source de la commande.

10 Logs dans un fichier

Dans un contexte de supervision, les logs doivent être conservés.

À faire

— écrire les logs dans un fichier `robot.log`,
— conserver l'affichage des logs dans le terminal.

11 Travail attendu

Vous devez :
— utiliser `argparse` pour configurer le programme,
— supprimer tous les `print()` de debug,
— utiliser exclusivement le module `logging`,
— structurer les logs par module,
— fournir une configuration claire et documentée.

Conclusion Arguments et logs sont deux outils essentiels pour transformer un script en une application robotique robuste, configurable et maintenable.

12 Pour aller plus loin...

Vous pouvez notamment utiliser les logs pour faire de l'analyse et de l'apprentissage sur le comportement du robot.